

Perfecto. **149 filas y 149 party_id únicos** confirma que ya tenemos correctamente una sola observación positiva por persona para el horizonte de 3 meses. Además, los casos cubren todo 2025, de enero a diciembre, lo cual es favorable porque no estamos concentrados en un único período.

Ahora avanzaría al **case-control EDA**. Antes de seleccionar controles, primero validemos cómo están distribuidos esos 149 casos por mes y por número de cuentas activas.

1. Distribución de los 149 casos por mes

```
SELECT
  data_date,
  COUNT(*) AS case_count
FROM bu_frd.alert_positive_3m_first_observation_2025
GROUP BY data_date
ORDER BY data_date;
```

Queremos comprobar que no haya, por ejemplo, 80 de los 149 casos concentrados en un solo mes.

2. Crear una banda de cuentas activas

Usaría estas bandas:

0
1
2–3
4–5
6–10
11+

Son suficientemente amplias para no generar demasiados estratos pequeños.

```
DROP TABLE IF EXISTS bu_frd.alert_positive_3m_first_observation_2025_v2;
```

```
CREATE TABLE bu_frd.alert_positive_3m_first_observation_2025_v2
USING PARQUET
AS
SELECT
  *,
```

```
  CASE
    WHEN active_accounts = 0
      THEN '00_ZERO'

    WHEN active_accounts = 1
      THEN '01_ONE'
```

```

    WHEN active_accounts BETWEEN 2 AND 3
      THEN '02_TWO_THREE'

    WHEN active_accounts BETWEEN 4 AND 5
      THEN '03_FOUR_FIVE'

    WHEN active_accounts BETWEEN 6 AND 10
      THEN '04_SIX_TEN'

    ELSE '05_ELEVEN_PLUS'

  END AS active_accounts_band

FROM bu_frd.alert_positive_3m_first_observation_2025;

Y revisa:

SELECT
  active_accounts_band,
  COUNT(*) AS case_count
FROM bu_frd.alert_positive_3m_first_observation_2025_v2
GROUP BY active_accounts_band
ORDER BY active_accounts_band;

```

3. Antes de muestrear controles: comprobar disponibilidad

No seleccionaría controles todavía. Primero queremos saber cuántos controles existen para cada combinación:

data_date + active_accounts_band

El pool de controles será:

```
DROP TABLE IF EXISTS bu_frd.alert_control_pool_3m_2025;
```

```
CREATE TABLE bu_frd.alert_control_pool_3m_2025
USING PARQUET
```

```
AS
```

```
SELECT
```

```
  l.*,
```

```
  CASE
```

```
    WHEN active_accounts = 0
      THEN '00_ZERO'
```

```
    WHEN active_accounts = 1
```

```

        THEN '01_ONE'

    WHEN active_accounts BETWEEN 2 AND 3
        THEN '02_TWO_THREE'

    WHEN active_accounts BETWEEN 4 AND 5
        THEN '03_FOUR_FIVE'

    WHEN active_accounts BETWEEN 6 AND 10
        THEN '04_SIX_TEN'

    ELSE '05_ELEVEN_PLUS'

END AS active_accounts_band

FROM bu_frd.party_trade_alert_labels_2025 l

WHERE alert_next_3m = 0

/* Excluir del pool a cualquiera de los 149 casos */
AND NOT EXISTS (

    SELECT 1
    FROM bu_frd.alert_positive_3m_first_observation_2025 p
    WHERE l.party_id = p.party_id

);

```

Esa exclusión es intencional. Aunque una persona que alerta posteriormente podría ser un negativo válido para una observación anterior, para este **primer EDA case-control** prefiero que casos y controles sean personas diferentes.

Ahora compara disponibilidad:

```

WITH cases AS (

    SELECT
        data_date,
        active_accounts_band,
        COUNT(*) AS case_count

    FROM bu_frd.alert_positive_3m_first_observation_2025_v2

    GROUP BY
        data_date,
        active_accounts_band

),

```

controls AS (

```
SELECT
    data_date,
    active_accounts_band,
    COUNT(*) AS available_control_count

FROM bu_frd.alert_control_pool_3m_2025
```

```
GROUP BY
    data_date,
    active_accounts_band
)
```

```
SELECT
    c.data_date,
    c.active_accounts_band,
    c.case_count,

    COALESCE(k.available_control_count,0)
        AS available_control_count,

    ROUND(
        1.0 * COALESCE(k.available_control_count,0)
        / c.case_count,
        2
    ) AS available_controls_per_case
```

FROM cases c

LEFT JOIN controls k

```
ON c.data_date = k.data_date
AND c.active_accounts_band = k.active_accounts_band
```

```
ORDER BY
    c.data_date,
    c.active_accounts_band;
```

Aquí quiero verificar que cada estrato tenga idealmente:

available_controls_per_case >= 20

Si algunas combinaciones no llegan a 20, no es grave; tomaremos todos los disponibles.

4. Seleccionar hasta 20 controles por caso

No necesitamos hacer un join caso-a-caso, porque nuestro matching es por estrato. Podemos seleccionar:

20×número de casos del estrato

controles de ese mismo estrato.

```
DROP TABLE IF EXISTS bu_frd.alert_controls_matched_3m_2025;
```

```
CREATE TABLE bu_frd.alert_controls_matched_3m_2025  
USING PARQUET  
AS
```

```
WITH case_stratum AS (
```

```
    SELECT  
        data_date,  
        active_accounts_band,  
        COUNT(*) AS case_count
```

```
    FROM bu_frd.alert_positive_3m_first_observation_2025_v2
```

```
    GROUP BY  
        data_date,  
        active_accounts_band
```

```
),
```

```
eligible_controls AS (
```

```
    SELECT  
        c.*,
```

```
        ROW_NUMBER() OVER (
```

```
            PARTITION BY  
                c.data_date,  
                c.active_accounts_band
```

```
            ORDER BY  
                hash(  
                    CAST(c.party_id AS STRING),  
                    CAST(c.data_date AS STRING)  
                )
```

```

        ) AS control_rank

FROM bu_frd.alert_control_pool_3m_2025 c
),

ranked AS (

    SELECT
        e.*,
        s.case_count

    FROM eligible_controls e

    INNER JOIN case_stratum s

        ON e.data_date = s.data_date
        AND e.active_accounts_band = s.active_accounts_band
    )

SELECT *
FROM ranked

WHERE control_rank <= case_count * 20;

```

Esto debería generar aproximadamente:

149 casos
~2,980 controles

aunque puede ser algo menor si algún estrato tiene pocos controles.

5. Validar el matching

```

SELECT
    'CASE' AS sample_type,
    COUNT(*) AS row_count,
    COUNT(DISTINCT party_id) AS party_count
FROM bu_frd.alert_positive_3m_first_observation_2025_v2

```

UNION ALL

```

SELECT
    'CONTROL' AS sample_type,
    COUNT(*) AS row_count,
    COUNT(DISTINCT party_id) AS party_count
FROM bu_frd.alert_controls_matched_3m_2025;

```

Y después verifica que la distribución temporal coincida:

```

SELECT
  sample_type,
  data_date,
  COUNT(*) AS row_count

FROM (

  SELECT
    'CASE' AS sample_type,
    data_date
  FROM bu_frd.alert_positive_3m_first_observation_2025_v2

  UNION ALL

  SELECT
    'CONTROL' AS sample_type,
    data_date
  FROM bu_frd.alert_controls_matched_3m_2025

) x

GROUP BY
  sample_type,
  data_date

ORDER BY
  data_date,
  sample_type;

```

No esperamos exactamente el mismo conteo —habrá aproximadamente 20 controles por caso—, pero sí la misma estructura mensual.

6. Crear el primer dataset case-control

Después:

```

DROP TABLE IF EXISTS bu_frd.alert_case_control_3m_2025;

CREATE TABLE bu_frd.alert_case_control_3m_2025
USING PARQUET
AS

SELECT
  *,
  1 AS case_flag
FROM bu_frd.alert_positive_3m_first_observation_2025_v2

```

UNION ALL

```
SELECT
  *,
  0 AS case_flag
FROM bu_frd.alert_controls_matched_3m_2025;
```

Aquí puede ocurrir que Spark no permita el **UNION ALL** porque las tablas de controles tienen columnas auxiliares como **control_rank** y **case_count**. Si sucede, no intentes forzarlo: seleccionaremos explícitamente las columnas necesarias. Lo podemos ajustar con base en el schema.

7. ¿Qué compararemos primero?

No empezaría con 30 features. Haría una primera prueba con nuestras **ocho señales principales**:

Feature	Qué representa
deteriorations_recent_3m	volumen de deterioro reciente
deterioration_acceleration	deterioro reciente vs baseline propio
adverse_recent_3m	entrada reciente a condición adversa
past_due_change_vs_baseline	cambio en deuda vencida
delinquency_share_change_vs_baseline	cambio en proporción de cuentas delinquent
balance_change_vs_baseline	cambio de exposición
utilization_change_vs_baseline	cambio en utilización revolving
max_dpd_jump_recent_3m	severidad máxima del deterioro

La comparación inicial será principalmente de **prevalencias, medianas y percentiles**, no de promedios, porque varias variables están extremadamente sesgadas.